

Phase 5 Notes: Dividing Functions with Constant Work

1. Current Progress Analysis

You are now at **Phase 5**.

Before this phase, you studied **decreasing recurrences**. In those problems, the input became smaller by subtraction.

Examples:

$T(n - 1)$
 $T(n - 2)$
 $T(n - b)$

That means the input changes like this:

$n \rightarrow n - 1 \rightarrow n - 2 \rightarrow n - 3 \rightarrow \dots$

This shrinking is slow because the input usually goes down one step at a time.

What you already learned

Phase	Main idea	Example recurrence	Final idea
Phase 1	Turn recursive code into a recurrence	$T(n) = T(n - 1) + 1$	Recurrence relations describe recursive running time.
Phase 2	Solve one-call decreasing recurrences	$T(n) = T(n - 1) + 1$	One decreasing call with constant work gives $O(n)$.
Phase 3	Study multiple decreasing calls	$T(n) = 2T(n - 1) + 1$	Two decreasing calls create a tree and give $O(2^n)$.
Phase 4	Use a shortcut for decreasing recurrences	$T(n) = aT(n - b) + f(n)$	Count recursive calls using a .

The big change in Phase 5

In Phase 5, the input no longer decreases by subtraction.

Instead, the input shrinks by **division**.

Example:

$n \rightarrow n / 2$

So instead of this slow pattern:

8 -> 7 -> 6 -> 5 -> 4 -> 3 -> 2 -> 1

we get this faster pattern:

8 -> 4 -> 2 -> 1

This is why Phase 5 gives us a new result:

$O(\log n)$

2. Main Goal of Phase 5

The main goal of Phase 5 is to understand recursive functions where the input is divided by 2 at each step.

The main recurrence is:

$T(n) = T(n / 2) + 1$

The final time complexity is:

$O(\log n)$

In simple words:

If a recursive function keeps cutting the input in half and does only a small constant amount of work each time, then the time complexity is $O(\log n)$.

3. What Is a Dividing Function?

A **dividing function** is a recursive function where the input becomes smaller by division.

Example recursive call:

```
algorithm1(n / 2);
```

This means the next function call receives half of the current input.

If $n = 16$, the calls go like this:

```
16 -> 8 -> 4 -> 2 -> 1
```

If $n = 32$, the calls go like this:

```
32 -> 16 -> 8 -> 4 -> 2 -> 1
```

The input becomes small very quickly.

4. Difference Between Decreasing and Dividing Recurrences

This difference is very important.

Type	Input change	Example	Growth idea
Decreasing recurrence	subtracts from n	$T(n - 1)$	Usually many levels
Dividing recurrence	divides n	$T(n / 2)$	Usually fewer levels

Decreasing example

```
T(n) = T(n - 1) + 1
```

Call pattern for $n = 8$:

```
8 -> 7 -> 6 -> 5 -> 4 -> 3 -> 2 -> 1 -> 0
```

This takes about n steps.

Final time:

$O(n)$

Dividing example

$T(n) = T(n / 2) + 1$

Call pattern for $n = 8$:

8 -> 4 -> 2 -> 1

This takes about $\log n$ steps.

Final time:

$O(\log n)$

5. Main Code Example

```
#include <iostream>
using namespace std;

void algorithm1(int n) {
    if (n > 1) {
        cout << n << " ";
        algorithm1(n / 2);
    }
}

int main() {
    algorithm1(8);
    return 0;
}
```

6. What the Code Does

The function is:

```
void algorithm1(int n) {  
    if (n > 1) {  
        cout << n << " ";  
        algorithm1(n / 2);  
    }  
}
```

It does three main things:

1. It checks whether `n > 1`.
2. If true, it prints the current value of `n`.
3. It calls itself with `n / 2`.

The function stops when `n` becomes `1` or smaller.

7. Important Parts of the Code

Part 1: Base condition

```
if (n > 1)
```

This means the function continues only while `n` is greater than `1`.

When `n = 1`, the condition becomes false:

```
1 > 1 is false
```

So the function stops.

The base case is:

```
T(1) = 1
```

This means the stopping case takes constant time.

Part 2: Current work

```
cout << n << " ";
```

This prints one value.

Printing one value takes constant time.

So the current work is:

```
+1
```

Part 3: Recursive call

```
algorithm1(n / 2);
```

The function calls itself once.

The input changes from:

```
n
```

to:

```
n / 2
```

So the recursive part is:

```
T(n / 2)
```

8. Writing the Recurrence Relation

The total time is:

```
current work + recursive call time
```

Current work:

1

Recursive call time:

$T(n / 2)$

So the recurrence is:

$T(n) = T(n / 2) + 1$

Base case:

$T(1) = 1$

Full recurrence:

$T(n) = T(n / 2) + 1$, for $n > 1$
 $T(1) = 1$

9. Meaning of Each Part of the Recurrence

The recurrence is:

$T(n) = T(n / 2) + 1$

Part	Meaning
$T(n)$	Time taken for input size n
$T(n / 2)$	Time taken by the recursive call on half the input
$+1$	Constant work done in the current call
$T(1) = 1$	Base case takes constant time

In simple words:

The function does one small job now, then solves the same problem for half the input.

10. General Form of a Division Recurrence

Dividing recurrences usually follow this form:

$$T(n) = aT(n / b) + f(n)$$

Where:

Symbol	Meaning
a	Number of recursive calls
b	Division factor
f(n)	Extra work done outside recursion

For Phase 5:

$$T(n) = T(n / 2) + 1$$

So:

Symbol	Value	Reason
a	1	There is one recursive call.
b	2	The input is divided by 2.
f(n)	1	The function prints once, which is constant work.

So the recurrence can also be written as:

$$T(n) = 1T(n / 2) + 1$$

Simplified:

$$T(n) = T(n / 2) + 1$$

11. Dry Run of `algorithm1(8)`

Call:

```
algorithm1(8);
```

Step-by-step table:

Call	Condition	Work done	Next call
<code>algorithm1(8)</code>	<code>8 > 1</code> is true	print <code>8</code>	<code>algorithm1(4)</code>
<code>algorithm1(4)</code>	<code>4 > 1</code> is true	print <code>4</code>	<code>algorithm1(2)</code>
<code>algorithm1(2)</code>	<code>2 > 1</code> is true	print <code>2</code>	<code>algorithm1(1)</code>
<code>algorithm1(1)</code>	<code>1 > 1</code> is false	stop	no call

Output:

```
8 4 2
```

The working calls are:

```
algorithm1(8)
algorithm1(4)
algorithm1(2)
```

There are only 3 working calls.

12. Why the Number of Calls Is Small

When `n = 8`, the input changes like this:

```
8 -> 4 -> 2 -> 1
```

When `n = 16`, the input changes like this:

```
16 -> 8 -> 4 -> 2 -> 1
```

When $n = 32$, the input changes like this:

32 -> 16 -> 8 -> 4 -> 2 -> 1

Each step cuts the number in half.

So the number of steps grows slowly.

This is called **logarithmic growth**.

13. Solving the Recurrence by Successive Substitution

We solve:

$$T(n) = T(n / 2) + 1$$

Step 1: Start with the recurrence

$$T(n) = T(n / 2) + 1$$

Step 2: Substitute $T(n / 2)$

Using the same rule:

$$T(n / 2) = T(n / 4) + 1$$

Substitute into the original recurrence:

$$T(n) = [T(n / 4) + 1] + 1$$

So:

$$T(n) = T(n / 4) + 2$$

Step 3: Substitute again

Using the same rule:

$$T(n / 4) = T(n / 8) + 1$$

So:

$$T(n) = [T(n / 8) + 1] + 2$$

Therefore:

$$T(n) = T(n / 8) + 3$$

Step 4: See the pattern

After 1 step:

$$T(n) = T(n / 2) + 1$$

After 2 steps:

$$T(n) = T(n / 4) + 2$$

After 3 steps:

$$T(n) = T(n / 8) + 3$$

Since:

$$\begin{aligned} 2 &= 2^1 \\ 4 &= 2^2 \\ 8 &= 2^3 \end{aligned}$$

After k steps:

$$T(n) = T(n / 2^k) + k$$

This is the general pattern.

14. Reaching the Base Case

The base case is:

$$T(1) = 1$$

So we want the input inside $T(\dots)$ to become 1 .

The pattern is:

$$T(n) = T(n / 2^k) + k$$

We need:

$$n / 2^k = 1$$

Now solve this equation.

Multiply both sides by 2^k :

$$n = 2^k$$

Now take logarithm base 2:

$$k = \log_2(n)$$

So the recursion reaches the base case after about:

$$\log_2(n)$$

steps.

15. Substitute Back

The pattern is:

$$T(n) = T(n / 2^k) + k$$

We found:

$$k = \log_2(n)$$

Substitute it back:

$$T(n) = T(1) + \log_2(n)$$

Since:

$$T(1) = 1$$

we get:

$$T(n) = 1 + \log_2(n)$$

In Big O notation, constants do not matter.

So:

$$T(n) = O(\log n)$$

16. Full Solution in One Place

Start:

$$T(n) = T(n / 2) + 1$$

Substitute:

$$T(n) = T(n / 4) + 2$$

Substitute again:

$$T(n) = T(n / 8) + 3$$

Pattern:

$$T(n) = T(n / 2^k) + k$$

Use the base case:

$$n / 2^k = 1$$

Solve:

$$\begin{aligned} n &= 2^k \\ k &= \log_2(n) \end{aligned}$$

Substitute back:

$$T(n) = T(1) + \log_2(n)$$

Since $T(1) = 1$:

$$T(n) = 1 + \log_2(n)$$

Final Big O:

$$T(n) = O(\log n)$$

17. What Does $\log n$ Mean Here?

In this phase, $\log n$ means:

How many times can we divide n by 2 until we reach 1?

Examples:

n	Dividing path	Number of divisions	$\log_2(n)$
8	8 -> 4 -> 2 -> 1	3	3
16	16 -> 8 -> 4 -> 2 -> 1	4	4
32	32 -> 16 -> 8 -> 4 -> 2 -> 1	5	5
64	64 -> 32 -> 16 -> 8 -> 4 -> 2 -> 1	6	6

So:

$\log_2(8) = 3$
 $\log_2(16) = 4$
 $\log_2(32) = 5$
 $\log_2(64) = 6$

The logarithm tells us the number of halving steps.

18. Beginner-Friendly Meaning of the Solution

The recurrence:

$$T(n) = T(n / 2) + 1$$

means:

Do 1 unit of work now, then solve the same problem for half the input.

The calls go like this:

n
n / 2
n / 4
n / 8
n / 16
...
1

At each level, the function does only constant work:

$$1 + 1 + 1 + 1 + \dots$$

But the number of levels is not n .

The number of levels is:

$$\log_2(n)$$

So the total work is:

$$O(\log n)$$

19. Time Complexity

The time complexity is:

$$O(\log n)$$

Why?

Because the function keeps cutting the input in half:

$$n \rightarrow n / 2 \rightarrow n / 4 \rightarrow n / 8 \rightarrow \dots \rightarrow 1$$

The number of times this can happen is about:

$$\log_2(n)$$

Each call does constant work.

So:

$$\text{constant work} \times \log n \text{ levels} = O(\log n)$$

20. Stack Space Complexity

The recursion stack space is also:

$O(\log n)$

Why?

The function calls form one chain:

```
algorithm1(n)
algorithm1(n / 2)
algorithm1(n / 4)
algorithm1(n / 8)
...
algorithm1(1)
```

The longest chain has about $\log n$ calls.

So the stack space is:

$O(\log n)$

21. Time Complexity vs Stack Space

For Phase 5:

Recurrence	Time complexity	Stack space
$T(n) = T(n / 2) + 1$	$O(\log n)$	$O(\log n)$

Why time is $O(\log n)$:

- The input is divided by 2 each time.
- There are about $\log n$ working calls.
- Each call does constant work.

Why space is $O(\log n)$:

- The recursive calls form one chain.
- The chain length is about $\log n$.

22. Complete Coding Program

```
#include <iostream>
using namespace std;

// This function prints n, then calls itself with n / 2.
void algorithm1(int n) {
    if (n > 1) {
        cout << n << " ";    // constant work: O(1)
        algorithm1(n / 2);    // recursive call on half the input
    }
}

int main() {
    int n = 8;

    cout << "Output: ";
    algorithm1(n);
    cout << endl;

    return 0;
}
```

Output:

Output: 8 4 2

Recurrence:

$$T(n) = T(n / 2) + 1$$
$$T(1) = 1$$

Time complexity:

$O(\log n)$

Stack space:

$O(\log n)$

23. Important Terms in Phase 5

Dividing function

A recursive function where the input becomes smaller by division.

Example:

```
algorithm1(n / 2);
```

Division recurrence

A recurrence where the recursive part contains division.

Example:

$$T(n) = T(n / 2) + 1$$

Constant work

Work that takes the same amount of time no matter how large `n` is.

Example:

```
cout << n << " ";
```

This is written as:

```
+1
```

Base case

The stopping condition for recursion.

In this phase:

```
if (n > 1)
```

The function stops when:

```
n = 1
```

So:

```
T(1) = 1
```

Logarithmic time

A time complexity where the number of steps grows by repeated division.

Example:

```
O(log n)
```

This means the algorithm grows slowly as `n` gets bigger.

Successive substitution

A method where we repeatedly replace the smaller recurrence until we find a pattern.

Example:

```
T(n) = T(n / 2) + 1  
T(n) = T(n / 4) + 2  
T(n) = T(n / 8) + 3  
T(n) = T(n / 2^k) + k
```

24. Common Beginner Mistakes

Mistake 1: Thinking `T(n / 2)` **means** `O(n / 2)`

Wrong idea:

$$T(n) = T(n / 2) + 1 \text{ gives } O(n / 2)$$

Correct idea:

$$T(n) = T(n / 2) + 1 \text{ gives } O(\log n)$$

Why?

Because the function does not divide only once.

It keeps dividing:

$$n \rightarrow n / 2 \rightarrow n / 4 \rightarrow n / 8 \rightarrow \dots \rightarrow 1$$

Repeated halving gives logarithmic time.

Mistake 2: Mixing up $T(n - 1)$ **and** $T(n / 2)$

These are different.

$T(n - 1)$ means:

$$n \rightarrow n - 1 \rightarrow n - 2 \rightarrow n - 3 \rightarrow \dots$$

This is slow shrinking.

$T(n / 2)$ means:

$$n \rightarrow n / 2 \rightarrow n / 4 \rightarrow n / 8 \rightarrow \dots$$

This is fast shrinking.

So:

$$\begin{aligned} T(n - 1) + 1 &= O(n) \\ T(n / 2) + 1 &= O(\log n) \end{aligned}$$

Mistake 3: Forgetting the base case equation

In Phase 2, the base case equation was:

$$n - k = 0$$

In Phase 5, the base case equation is:

$$n / 2^k = 1$$

This equation is very important because it gives:

$$k = \log_2(n)$$

Mistake 4: Assuming all recursive functions are $O(n)$

Not all recursive functions are $O(n)$.

Examples:

Recurrence	Final time
$T(n) = T(n - 1) + 1$	$O(n)$
$T(n) = 2T(n - 1) + 1$	$O(2^n)$
$T(n) = T(n / 2) + 1$	$O(\log n)$

You must always check:

1. How many recursive calls are made?
2. How does the input change?
3. How much extra work happens outside recursion?

Mistake 5: Using the wrong master theorem

Do not use the decreasing master theorem for this phase.

The decreasing master theorem is for:

$T(n - b)$

Phase 5 is about:

$T(n / b)$

These are different recurrence families.

Mistake 6: Forgetting stack space

Even though the time is $O(\log n)$, the function also uses recursive stack memory.

The stack space is:

$O(\log n)$

because the deepest chain has about $\log n$ calls.

25. Step-by-Step Checklist for Phase 5 Problems

When you see a dividing recursive function, follow these steps.

Step 1: Find the base case

Look for the stopping condition.

Example:

```
if (n > 1)
```

The function stops when:

$n = 1$

So:

$$T(1) = 1$$

Step 2: Count recursive calls

Example:

```
algorithm1(n / 2);
```

There is one recursive call.

So:

```
a = 1
```

Step 3: Track how the input changes

The input changes from:

```
n
```

to:

```
n / 2
```

So the recursive part is:

```
T(n / 2)
```

Step 4: Count extra work

Example:

```
cout << n << " ";
```

This prints once.

So the extra work is:

+1

Step 5: Write the recurrence

$$\begin{aligned}T(n) &= T(n / 2) + 1 \\T(1) &= 1\end{aligned}$$

Step 6: Substitute repeatedly

$$\begin{aligned}T(n) &= T(n / 2) + 1 \\T(n) &= T(n / 4) + 2 \\T(n) &= T(n / 8) + 3\end{aligned}$$

Step 7: Find the pattern

$$T(n) = T(n / 2^k) + k$$

Step 8: Use the base case

$$n / 2^k = 1$$

Step 9: Solve for k

$$\begin{aligned}n &= 2^k \\k &= \log_2(n)\end{aligned}$$

Step 10: Write the final Big O

$$T(n) = O(\log n)$$

26. Mini Practice Example 1

Find the recurrence and time complexity.

```
void test(int n) {  
    if (n > 1) {  
        cout << "*";  
        test(n / 2);  
    }  
}
```

Solution

Base case:

$$T(1) = 1$$

There is one recursive call:

```
test(n / 2);
```

So the recursive part is:

$$T(n / 2)$$

The current work is printing once:

+1

Recurrence:

$$T(n) = T(n / 2) + 1$$

Time complexity:

$O(\log n)$

Stack space:

$O(\log n)$

27. Mini Practice Example 2

Find the recurrence and time complexity.

```
void countHalf(int n) {  
    if (n > 1) {  
        cout << n;  
        countHalf(n / 2);  
    }  
}
```

Solution

The function prints once, so current work is:

+1

The function calls itself once with half the input:

$T(n / 2)$

Recurrence:

$T(n) = T(n / 2) + 1$
 $T(1) = 1$

Final time:

$O(\log n)$

Stack space:

$O(\log n)$

28. Simple Way to Remember Phase 5

For this code pattern:

```
function(n) {  
  if (n > 1) {  
    do one small work;  
    function(n / 2);  
  }  
}
```

The recurrence is:

$T(n) = T(n / 2) + 1$

The time complexity is:

$O(\log n)$

The stack space is:

$O(\log n)$

Memory trick:

Subtract by 1 -> linear chain -> $O(n)$
Divide by 2 -> logarithmic chain -> $O(\log n)$

Another memory trick:

$\log n$ means number of times we can keep cutting n in half until we reach 1.

29. Phase 5 Final Summary

In Phase 5, you learned about dividing functions with constant work.

The main code was:

```
void algorithm1(int n) {  
    if (n > 1) {  
        cout << n << " ";  
        algorithm1(n / 2);  
    }  
}
```

The function:

1. Prints the current value of n .
2. Calls itself with $n / 2$.
3. Stops when $n = 1$.

The recurrence is:

$$\begin{aligned}T(n) &= T(n / 2) + 1 \\T(1) &= 1\end{aligned}$$

Using substitution:

$$\begin{aligned}T(n) &= T(n / 2) + 1 \\T(n) &= T(n / 4) + 2 \\T(n) &= T(n / 8) + 3\end{aligned}$$

Pattern:

$$T(n) = T(n / 2^k) + k$$

Base case equation:

$$n / 2^k = 1$$

Solve:

$$k = \log_2(n)$$

Final result:

$$T(n) = O(\log n)$$

Stack space:

$$O(\log n)$$

The most important lesson is:

Repeatedly dividing the input by 2 creates logarithmic time.

30. What You Should Be Able to Do After Phase 5

After Phase 5, you should be able to:

1. Recognize a dividing recursive function.
 2. Understand that $T(n / 2)$ is different from $T(n - 1)$.
 3. Identify the base case for a dividing function.
 4. Count the number of recursive calls.
 5. Identify that `algorithm1(n / 2)` gives the recursive term $T(n / 2)$.
 6. Identify constant work as `+1`.
 7. Write the recurrence $T(n) = T(n / 2) + 1$.
 8. Solve the recurrence using successive substitution.
 9. Find the pattern $T(n) = T(n / 2^k) + k$.
 10. Use the base case equation $n / 2^k = 1$.
 11. Understand why $k = \log_2(n)$.
 12. Conclude that the time complexity is $O(\log n)$.
 13. Understand why the stack space is also $O(\log n)$.
 14. Avoid common mistakes like saying the answer is $O(n / 2)$.
 15. Explain in simple words why repeated halving gives logarithmic time.
-

31. Key Formulas From Phase 5

Main recurrence:

$$\begin{aligned}T(n) &= T(n / 2) + 1 \\T(1) &= 1\end{aligned}$$

After substitution:

$$T(n) = T(n / 2^k) + k$$

Base case equation:

$$n / 2^k = 1$$

Solve:

$$\begin{aligned}n &= 2^k \\k &= \log_2(n)\end{aligned}$$

Final result:

$$\begin{aligned}\text{Time: } &O(\log n) \\ \text{Space: } &O(\log n)\end{aligned}$$

Most important memory sentence:

If one recursive call cuts n in half and does constant work, the answer is $O(\log n)$.